

What happens to your
program when it grows
from 200 lines to 20,000
lines?



Congratulations! Your
program works... until
you change one line.

If we model the real world with data *and* actions together, why do we separate them in our programs?



Introduction to Object Oriented Programming

Object Oriented Programming

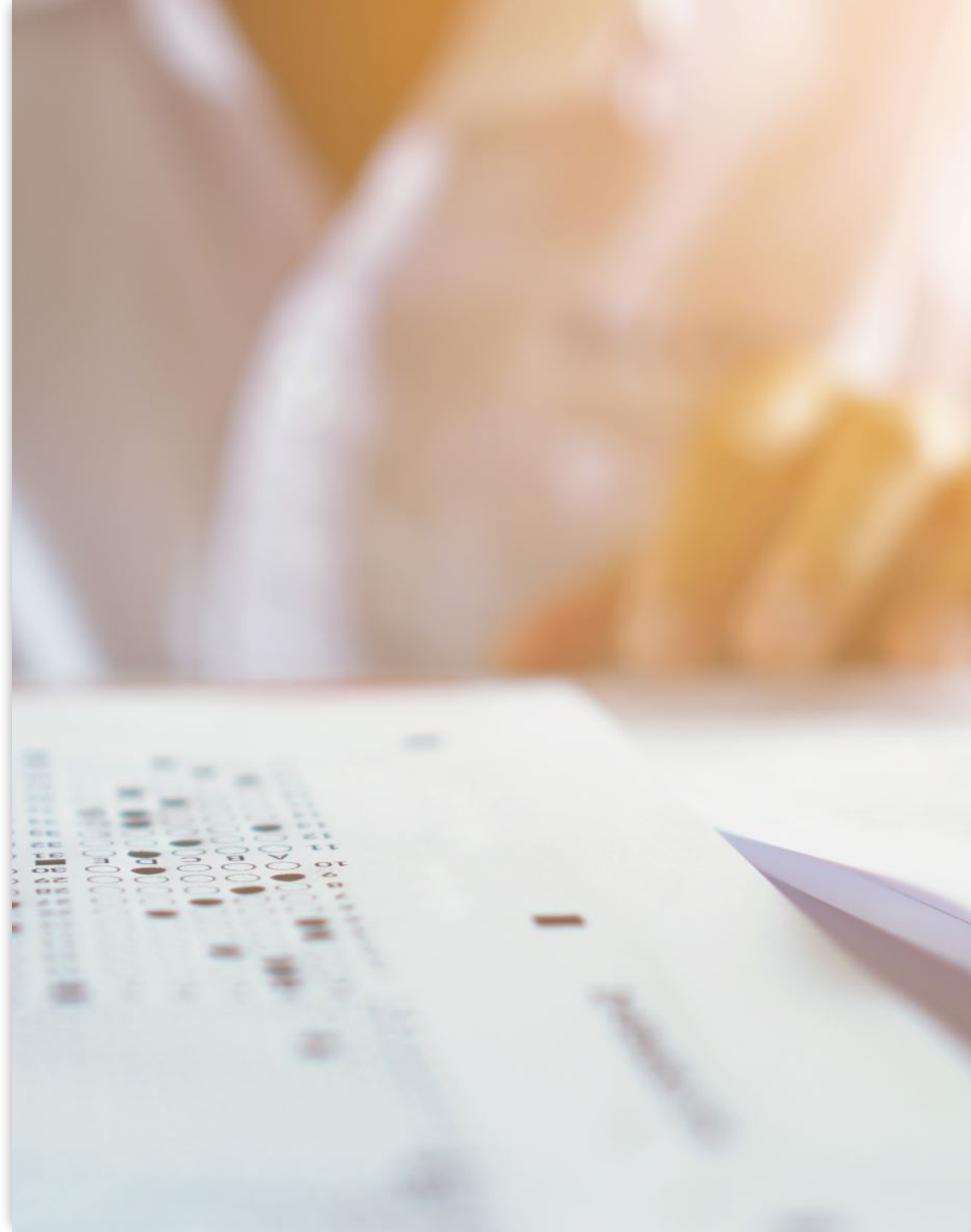
By

Bilal Ahmed



Introduction to Course

- Assessment (Theory 100 Marks)
 - 30% Mid Term
 - 30% Sessional:
 - 50% Project
 - 50% Quiz and Assignments
- Course Outlines
- 75% Attendance is mandatory for appearing in Final Term Exam, and there will be no compromise on attendance.
- Assignments Should be submitted on due time.
- Quiz will not be retaken.
- During the class, if any one found using cellphone, will get penalty.



Procedural Programming – Quick Recap

Procedural Programming

- Program is organized around **functions**
- Data is stored separately (often global or shared)
- Functions operate on the same data
- Focus: **“What steps should the program follow?”**

Common Issues

- Hard to manage as programs grow
- Data can be modified by many functions
- Changes in one place affect many others

Object-Oriented Programming – Big Idea



Program is organized around **objects**



Objects combine **data + behavior**



Each object manages its own data



Focus: “**Who is responsible for what?**”

Procedural vs Object-Oriented (Comparison)

Procedural Programming	Object-Oriented Programming
Functions are central	Objects are central
Data is separate from functions	Data and methods are bundled
Uses functions & procedures	Uses classes & objects
Harder to maintain large systems	Easier to maintain & extend
Less real-world modeling	Models real-world entities



Key Difference

Procedural:

*Functions act on
data*

Object-Oriented:

*Objects own their
data and behavior*

Conceptual Example – Student (Procedural)

Data

- name
- rollNo
- marks

Functions

- calculateGrade(student)
- displayStudent(student)
- updateMarks(student)

Data passed around to multiple functions

Conceptual Example – Student (OOP)

Student Object

- **Attributes:** name, rollNo, marks
- **Methods:** calculateGrade(), displayInfo(), updateMarks()

Data + behavior inside one object

What OOP Is NOT



Not just “using classes”



Not language-specific

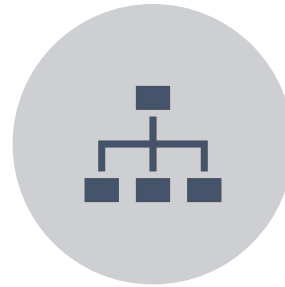


Not magic — it’s a way of thinking/designing

Summary



Procedural programming works well for **small programs**



Large systems need better **structure and organization**



OOP helps manage complexity using **objects**



This is why modern software prefers **OOP**

Programming Languages and OOP Support

Supports Object-Oriented Programming

- Java
- C++
- Python
- C#

Does NOT Support Object-Oriented Programming

- C
- Assembly Language
- Early BASIC
- Old Fortran



So should we use OOP
everywhere?

Where Object-Oriented Programming Is Used

Where Object-Oriented Programming Is Used

- Large software systems
- Web applications
- Mobile applications
- Desktop applications
- Game development
- Enterprise systems (banking, ERP, management software)

Why?

- Better structure
- Easier maintenance
- Scales well as projects grow

Where Object-Oriented Programming Is NOT Preferred

Where Object-Oriented Programming Is NOT Preferred

- Small or simple programs
- Low-level system programming
- Embedded systems with limited memory
- Performance-critical code (device drivers, kernels)
- Scripting or quick automation tasks

Why?

- Extra overhead
- More complexity than needed

Structural (Procedural) Programming

Advantages

- Simple and easy to understand
- Faster to write for **small programs**
- Low memory overhead
- Good performance
- Easy to debug in small codebases

Disadvantages

- Poor scalability for large programs
- Data is often shared or global
- Changes in one function can affect many others
- Code reuse is limited
- Hard to maintain as complexity grows

Object-Oriented Programming (OOP)

Advantages

- Models real-world problems naturally
- Better **code organization**
- High **reusability** (classes, inheritance)
- Easier maintenance and extension
- Supports modular and scalable design

Disadvantages

- More complex to learn initially
- Requires more planning and design
- Slight performance and memory overhead
- Overkill for very small programs

Real life applications

Procedural Programming

- **Embedded systems** (Arduino, microcontrollers)
- **IoT devices** (sensors, controllers)
- **Operating systems & drivers**
- **Small scripts and utilities**

Object-Oriented Programming (OOP)

- **Banking & enterprise systems**
- **Web applications** (e-commerce, social media)
- **Mobile & desktop applications**
- **Game development**



Why should cyber security
specialists learn this?

OOP in Cybersecurity – Where It's Used

**OOP is widely used
in cybersecurity
tools such as:**

- Antivirus and malware scanners
- Intrusion Detection / Prevention Systems (IDS/IPS)
- Firewall and network monitoring tools
- Vulnerability scanners
- Security Information and Event Management (SIEM) systems

Why OOP?

- Many entities to model (users, packets, rules, threats)
- Complex and evolving systems
- Easy to extend with new attack types or rules

Example 1: Antivirus Software

Objects:

- File
- MalwareSignature
- Scanner
- Threat

Each object:

- Stores its own data
- Has its own behavior (scan, detect, quarantine)

Example 2: Network Security / IDS

Objects:

- Packet
- Connection
- Rule
- Alert

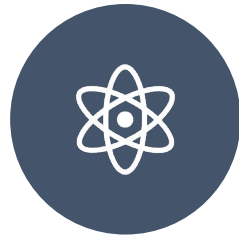
Benefits:

- Rules can be added without rewriting the system
- Different attack types handled via polymorphism

Core OOP Terminology (Basic)



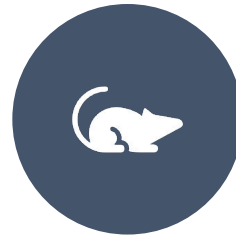
OBJECT



CLASS
(BLUEPRINT)



ATTRIBUTES
(STATE)



METHODS
(BEHAVIOR)



INSTANCE

Class (Blueprint)

A **class** is a blueprint or template

It defines two things

- What data an object will have
- What actions it can perform

No memory is used until objects are created

A **NetworkPacket** class defines what a packet looks like.

Object

An **object** is a real entity created from a class

Represents one specific thing

Has its own data and behavior

A **specific packet** captured from the network.

Attributes (State)

Attributes are **data members** of a class

They represent the **state** of an object

Usually variables are

- source IP
- destination IP
- protocol

Methods (Behavior)

Methods are **functions** inside a class

Define what an object can do

Operate on the object's own data

Example methods:

- `checkProtocol()`
- `displayPacket()`
- `isSuspicious()`

Instance



An **instance** is a specific object created from a class



Multiple instances can be created from one class



Each instance has its own data



Eg. Each captured network packet is an instance of NetworkPacket.

C++ Code Example

```
#include <iostream>
using namespace std;

// Class (Blueprint)
class NetworkPacket {
public:
    // Attributes (State)
    string sourceIP;
    string destinationIP;
    string protocol;

    // Method (Behavior)
    void displayPacket() {
        cout << "Source: " << sourceIP << endl;
        cout << "Destination: " << destinationIP << endl;
        cout << "Protocol: " << protocol << endl;
    }
};
```

C++ Code Example

```
int main() {  
    // Object / Instance  
    NetworkPacket packet1;  
  
    // Setting attributes  
    packet1.sourceIP = "192.168.1.10";  
    packet1.destinationIP = "8.8.8.8";  
    packet1.protocol = "TCP";  
  
    // Calling method  
    packet1.displayPacket();  
  
    return 0;  
}
```


Summary

Class → Blueprint (NetworkPacket)

Object / Instance → Real packet captured

Attributes → Packet details (IP, protocol)

Methods → Actions on packet (analyze, display)

In cybersecurity, everything we
analyze — packets, users, logs, alerts
— can be modeled as objects.
